

Sub-module Introduction

Mladen Ivkovic

**COMP2221 Programming Paradigms –
Object-oriented Programming**

Module Tutors

- Mladen Ivkovic
 - Career Development Fellow in Computer Science
 - **Research interests:** Scientific Computing (Astrophysics, Cosmology, Fluid Dynamics), High Performance Computing
 - mladen.ivkovic@durham.ac.uk



Office hours

- See Blackboard, and email me to make any enquiry
- We can always chat at the end of the lecture!

Teaching Arrangements

- Lectures
 - Theory background, object-oriented programming concepts, programming knowledge, demonstrations
- Workshops
 - Guided programming tasks, tutorials

Blackboard Site

- https://blackboard.durham.ac.uk/ultra/courses/_67655_1/outline
Scheduled release for lecture notes and practicals

The screenshot shows the Blackboard course page for 'Programming Paradigms (25/26) COMP2221_2025'. The page has a dark purple header with the course title. Below the header, there is a 'Course Content' section with a search icon. It contains five items, each with a document icon and a dropdown arrow: 'Start Here' (Learn about this module's structure, how it will be taught and the activities designed for you.), 'Assessment', 'Systems Programming' (Term 1: This submodule is taught twice a week by Stuart James. Lectures take place Monday 2pm in CLC202 and Friday 10am in CLC202. The submodule is assessed by coursework in the Winter.), 'Functional Programming' (Term 2: This submodule is taught twice a week in the first half of Term 2 by Max Gadouleau. Lectures take place Monday 2pm in CLC202 and Friday 10am in CLC202. The submodule is assessed by exam in the summer.), and 'Object Oriented Programming' (Term 2: This submodule is taught twice a week in the second half of Term 2 by Miladen Ivkovic. Lectures take place Monday 2 pm in CLC202 and Friday 10am in CLC202. The submodule is assessed by exam in the summer.). To the right of the 'Course Content' section is a 'Course Staff' section with two instructors: Maximilien Gad... and Miladen Ivkovic, each with a profile picture, name, role (INSTRUCTOR), and an email icon. Below the staff list is a 'Show more' link. Further down is a 'Details & Actions' section with three items: 'Class register' (with a link 'View everyone on your course'), 'Books & Tools' (with a link 'View course & Institution tools'), and 'Microsoft Education' (with a link 'Enable Microsoft Education'). A help icon (?) is located at the bottom right of the page.

What to Expect in This Sub-module?

- Object-oriented programming (OOP)
 - Theory and practical implementation
- A new programming language – Java
- A Development Environment (IDE) – Eclipse
- I hope you have some fun too!

Object-oriented Programming (OOP)

- A programming framework that facilitates the reuse and extensibility of code
 - With OOP, source code becomes easier to maintain and follow
 - Essential for larger-scale software development
- A naïve comparison:

Non-OOP	OOP
A long sequence of source code	Bits of source code with relevant purposes are grouped together as <i>classes</i> , managed as a hierarchy with <i>inheritance</i>
C	Java, C#

Java

- Sufficiently similar to both C++ and C#
 - If you know Java, you should have no problem with C# and C++
- An *object-oriented, cross-platform* programming language
 - Well-designed to integrate OOP concepts
 - Remains one of the most popular programming languages
 - Usable in Windows, macOS, and Linux with the right compiler/environment installed



Topics Covered

- You will learn
 - Java programming
 - OOP with Java
 - Encapsulation
 - Inheritance
 - Abstraction
 - Polymorphism
 - Advanced programming topics
- Note that the teaching materials are new
 - If you spot any errors, please let me know 😊

Sub-module Assessment

- 100% based on an exam
 - 3 questions
 - Assessing OOP concepts and Java programming & implementation

Module Textbooks

- "Effective Java" by Joshua Bloch
 - Available in [library](#)
- Documentation
 - <https://docs.oracle.com/en/java/javase/25/>
- Tutorials
 - <https://www.oracle.com/java/technologies/jdtt-jsp.html>
- Other online resources as indicated in the reference list of each lecture



Summary

- Object-oriented Programming (OOP)
- Java
- Eclipse



Any questions so far?



Java and Java Programming: Fundamentals

Exploring Object-Oriented Concepts, Classes, and Objects

What is Java?

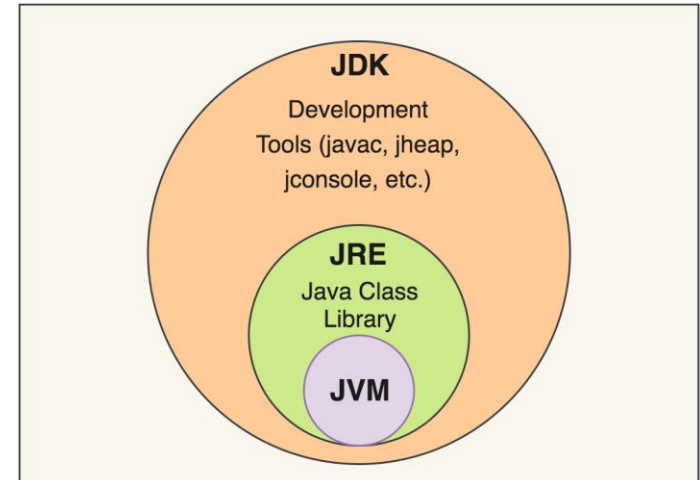
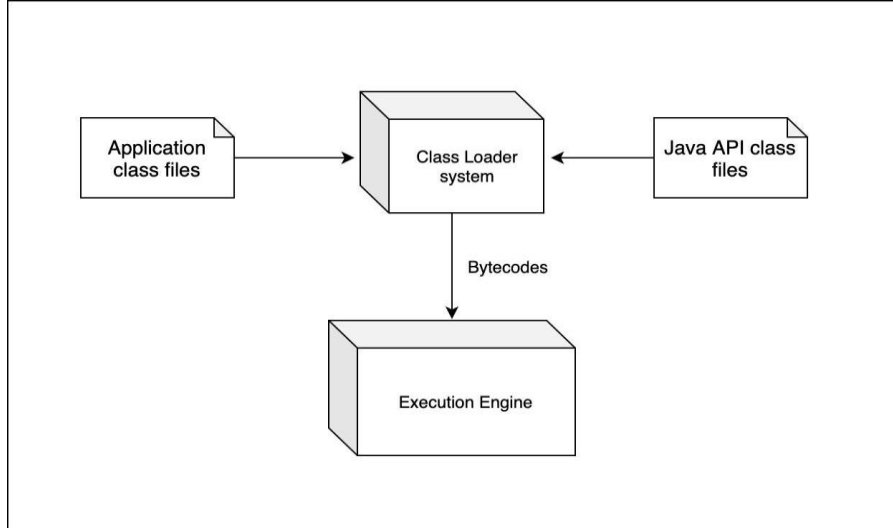
- Java is a high-level, object-oriented programming language.
- Developed by Sun Microsystems (now owned by Oracle) in 1995.
- Follows the principle of '*Write Once, Run Anywhere*' (WORA).
- Used in web applications, mobile development, and enterprise software.

Java as an Object-Oriented Language

- Based on
 - **Encapsulation**
 - **Inheritance**
 - **Polymorphism**
 - **Abstraction**
- **Everything in Java is an object.**
- Objects contain data and methods.
- **A class defines the blueprint for objects.**
- Promotes modular, reusable, and maintainable code.

Java Virtual Machine (JVM)

- JVM is a virtual machine that enables the execution of Java bytecode.
- The JVM acts as an interpreter between the Java programming language and the underlying hardware.
- It provides a *runtime environment* for Java applications to run on different platforms and operating systems.



What is a Class?

- A **class** is a template for creating objects.
- It defines **state (variables) and behavior (methods)**.
- A class does not occupy memory until an object is created.
- **Example:**

```
class Car {  
    // variables  
    String brand;  
    int speed;  
  
    // methods  
    void accelerate() {  
        speed += 10;  
    }  
}
```

What is an Object?

- An **object** is an instance of a class.
- It has **state (attributes) and behavior (methods)**.
- Objects are stored in memory at runtime.
- **Example of Creating an Object:**

```
public class TestCar {  
    public static void main(String[] args) {  
        Car myCar = new Car();  
        myCar.brand = "Toyota";  
        myCar.accelerate();  
        System.out.println("Brand: " + myCar.brand + " Speed: " + myCar.speed);  
    }  
}
```

Encapsulation in Java

- Encapsulation means **hiding internal details** and exposing only what is necessary.
- **Private variables** can only be accessed via **public methods (getters and setters)**.
- **Example:**

```
class Person {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String newName) {  
        name = newName;  
    }  
}
```

Inheritance in Java

- **Inheritance** allows a new class to inherit from an existing class.
- Promotes **code reusability** and **hierarchy**.
- **Example:**

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

Polymorphism in Java

- **Polymorphism allows a method to take different forms.**
- **Method Overriding:** A subclass provides a different implementation of a method from the parent class.
- **Example:**

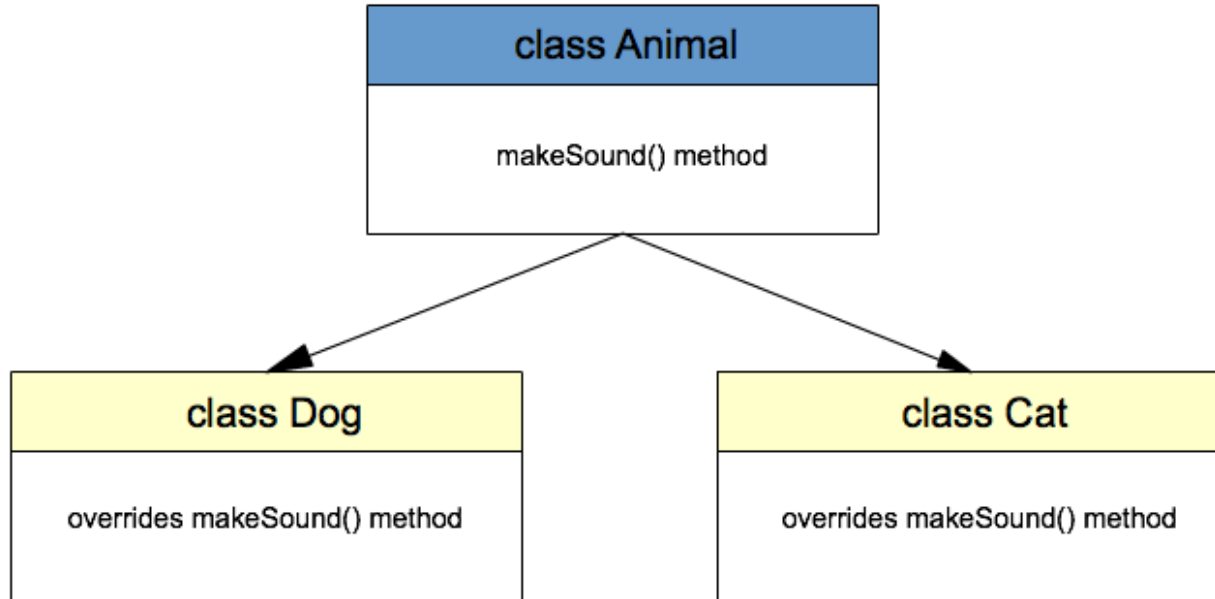
```
class Animal {  
    void makeSound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

Abstraction in Java

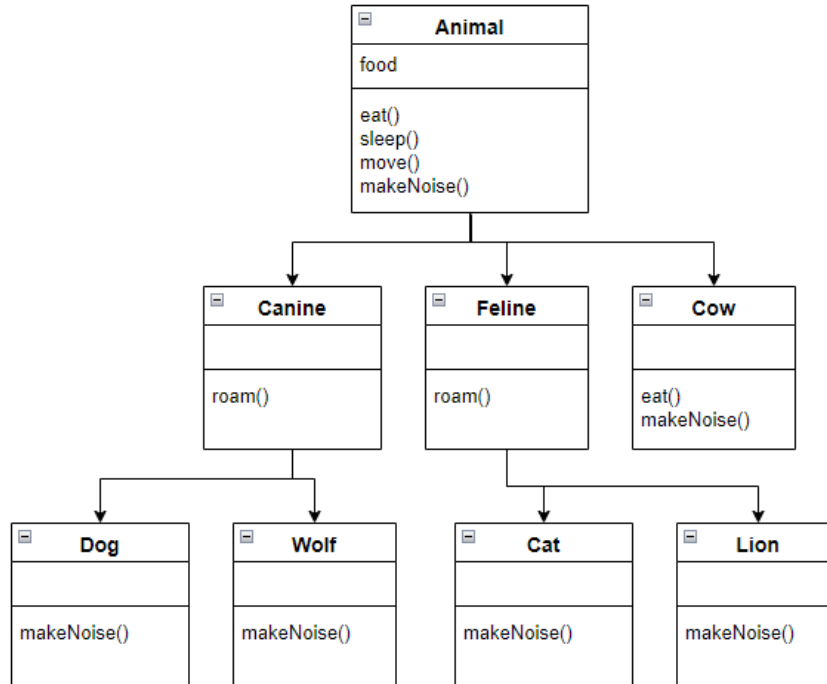
- **Abstraction hides implementation details and only shows relevant functionality.**
- Defines common structure, but not (necessarily) implementation
- Achieved using **abstract classes or interfaces.**
- **Example:**

```
abstract class Animal {  
    // No implementation!  
    abstract void makeSound();  
}  
  
class Cat extends Animal {  
    void makeSound() {  
        System.out.println("Meow");  
    }  
}
```

UML (Unified Modeling Language) and Java



UML and Java



Conclusion

- Java is a powerful **object-oriented programming language**.
- Key principles: **Encapsulation, Inheritance, Polymorphism, and Abstraction**.
- Java enables **modular, reusable, and scalable** application development.
- Next steps: Explore **Advanced Java concepts** like *Collections*, *Exception Handling*, and *Multi-threading*.

Any questions?

